

8

Sealed and Hidden Classes

This chapter includes 13 problems covering sealed and hidden classes. The first 11 recipes will cover sealed classes, a very cool feature introduced in JDK 17 (JEP 409) to sustain *closed hierarchies*. The last two problems cover hidden classes, a JDK 15 (JEP 371) feature that allows frameworks to create and use runtime (dynamic) classes hidden in the JVM's internal linkages of bytecode, and to the explicit usage of class loaders.

You'll be skilled in both topics by the end of this chapter.

Problems

Use the following problems to test your programming prowess in manipulating sealed classes and hidden classes in Java. I strongly encourage you to give each problem a try before you turn to the solutions and download the example programs:

172. **Creating an electrical panel (hierarchy of classes):** Write the stub of a Java application that shapes an electrical

panel. You can assume that the electrical panel is made of several types of electrical components (for instance, resistors, transistors, and so on), and electrical circuits (for instance, parallel circuits, series circuits, and so on).

173. **Closing the electrical panel before JDK 17:** Use the Java features (for instance, the `final` keyword and *package-private* hacks) to close this hierarchy (close to extension).
174. **Introducing JDK 17 sealed classes:** Provide a brief introduction to JDK 17 sealed classes. Exemplify how to write closed hierarchies in a single source file via sealed classes.
175. **Introducing the permits clause:** Explain and exemplify the role of the `permits` clause in sealed classes. Exemplify sealed classes in different source files (same package) and in different packages.
176. **Closing the electrical panel after JDK 17:** Use sealed classes to completely close the electrical panel hierarchy developed in problems 172 and 173.
177. **Combining sealed classes and records:** Exemplify the usage of Java records in combination with sealed classes.
178. **Hooking sealed classes and instanceof:** Write an application that highlights how sealed classes help the compiler to better handle the `instanceof` operator.
179. **Hooking sealed classes in switch:** Write an application that exemplifies how sealed classes help the compiler to sustain exhaustive switch expressions/statements.

- 180. Reinterpreting the Visitor pattern via sealed classes and type pattern matching for switch:** Provide a quick sample of the traditional Visitor pattern implementation and transform it via sealed classes into simpler and more accessible code.
- 181. Getting info about sealed classes (using reflection):** Explain and exemplify how we can access sealed classes via Java Reflection.
- 182. Listing the top three benefits of sealed classes:** Provide what you think are the top three benefits of sealed classes with a few explanations and arguments.
- 183. Briefly introducing hidden classes:** Provide a brief, crystal-clear, and meaningful explanation of hidden classes. List their main characteristics.
- 184. Creating a hidden class:** Provide a regular example of creating and using a hidden class.

The following sections describe solutions to the preceding problems. Remember that there usually isn't a single correct way to solve a particular problem. Also, remember that the explanations shown here include only the most interesting and important details needed to solve the problems. Download the example solutions to see additional details and to experiment with the programs at

<https://github.com/PacktPublishing/Java-Coding-Problems-Second-Edition/tree/main/Chapter08>.

172. Creating an electrical panel (hierarchy of classes)

Let's assume that we want to model in code lines an electrical panel. Of course, we are not electricians, so for our purposes, an electric panel means a box with some internal circuits made of electrical components and a breaker that turns on/off the electrical panel.

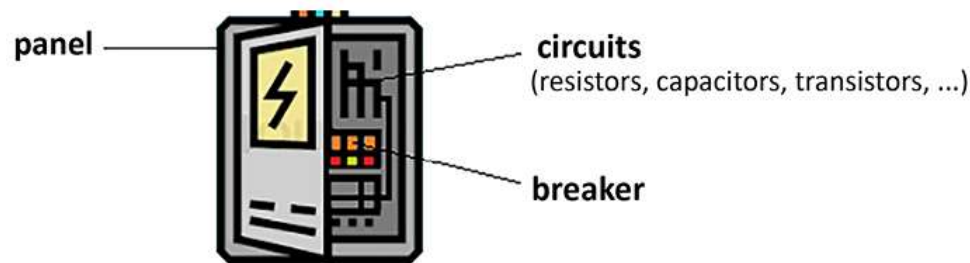


Figure 8.1: Electrical panel components

Everything in an electrical panel can be considered an electrical component, so we can start our code by defining an interface that must be implemented by everything in this panel:

```
public interface ElectricComponent {}
```

Before continuing, let's look at a diagram of the electric panel interfaces and classes that will help you to follow what comes

after more easily:

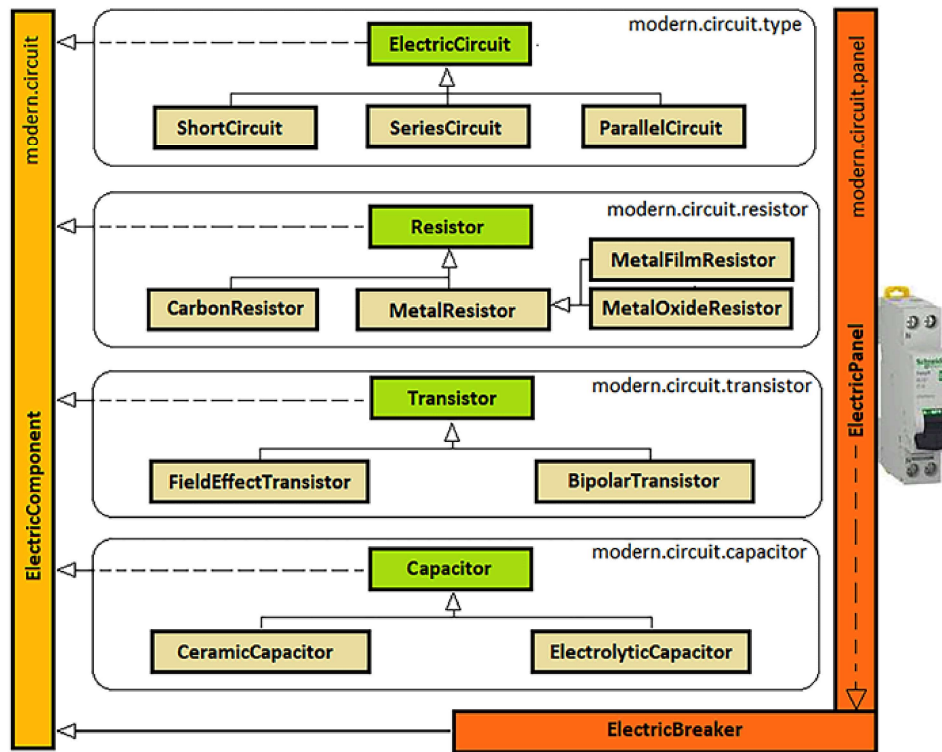


Figure 8.2: A model of the electrical panel

An electrical panel consists of more electrical circuits that interact (or do not interact) with each other. We can represent such a circuit via an `abstract` class as follows (this acts as a base class for its subclasses):

```
public abstract class ElectricCircuit
    implements ElectricComponent {
```

```
public abstract void on();  
public abstract void off();  
}
```

Let's assume that our electrical panel is made of three types of circuits. We have short circuits, series circuits, and parallel circuits. So, we can define the proper class for each type of circuit by extending the `abstract ElectricCircuit` (we'll show here only the `ShortCircuit`, while the `ParallelCircuit` and `SeriesCircuit` are available in the bundled code):

```
public class ShortCircuit extends ElectricCircuit {  
    public ShortCircuit(ElectricComponent... comp) {}  
    @Override  
    public void on() {}  
    @Override  
    public void off() {}  
}
```

Check out the constructor of the `ShortCircuit` class. It gets a `varargs` argument of the type `ElectricComponent`. This means that we can build a bigger circuit from smaller circuits and other components such as capacitors, transistors, resistors, and so on. Each such electrical component can be shaped via an `abstract` class. For instance, a capacitor is a base class that can be expressed as follows:

```
public abstract class Capacitor
    implements ElectricComponent {}
```

We need two types of capacitors (ceramic capacitors and electrolytic capacitors). A ceramic capacitor can be shaped as follows:

```
public class CeramicCapacitor extends Capacitor {}
```

Following the same logic, we can express other electrical components such as transistors (`Transistor` (`abstract`), `BipolarTransistor`, and `FieldEffectTransistor`) and resistors (`Resistor` (`abstract`), `CarbonResistor`, and `MetalResistor`, and its two subtypes, `MetalFilmResistor` and `MetalOxideResistor`).

We almost have all the electrical components required for building our panel. We just need the breaker, which is just another electrical component with the particularity that it exposes two methods for turning on/off the electrical panel:

```
public interface ElectricBreaker extends ElectricComponent {
    void switchOn();
    void switchOff();
}
```

And, finally, we can materialize the electrical panel as follows (we assume that we have three circuits, a central circuit, a peripheral circuit, and an auxiliary circuit):

```
public class ElectricPanel implements ElectricBreaker {
    private final ElectricCircuit centralCircuit;
    private final ElectricCircuit peripheralCircuit;
    private final ElectricCircuit auxiliaryCircuit;
    public ElectricPanel() {
        peripheralCircuit = new SeriesCircuit(
            new ElectrolyticCapacitor(), new ElectrolyticCapacitor(),
            new MetalFilmResistor(), new CarbonResistor());
        auxiliaryCircuit = new ShortCircuit(
            new CeramicCapacitor(), new ElectrolyticCapacitor(),
            new MetalResistor(), new FieldEffectTransistor(),
            new FieldEffectTransistor());
        centralCircuit = new ParallelCircuit(
            peripheralCircuit, auxiliaryCircuit,
            new CeramicCapacitor(), new BipolarTransistor(),
            new MetalOxideResistor());
    }
    @Override
    public void switchOn() {
        auxiliaryCircuit.off();
        peripheralCircuit.on();
        centralCircuit.on();
    }
    @Override
    public void switchOff() {
```

```
        auxiliaryCircuit.on();  
        peripheralCircuit.off();  
        centralCircuit.off();  
    }  
}
```

Done! Now, a client of our panel can operate it via the `switchOn()` / `switchOff()` methods:

```
ElectricPanel panel = new ElectricPanel();  
panel.switchOn();
```

In the next problem, we will see how we can close this hierarchy of classes in order to increase encapsulation and avoid accidental/non-accidental extensions.

173. Closing the electrical panel before JDK 17

By its nature, an electrical panel is a closed unit of work. But our code from the previous problem is far from being a closed hierarchy. We can extend and implement almost any class/interface from inside or outside the hierarchy.

Using anything before JDK 17, closing a hierarchy of classes and interfaces can be done using several tools.

Applying the final modifier

For instance, we have the powerful `final` modifier. Once we declare a class as `final`, it cannot be extended, so it is completely closed to extension. Obviously, we cannot apply this technique consistently across a hierarchical model because it will lead to a non-hierarchical model.

If we scan our electrical panel model, then we can use the `final` modifier in several places. First, we eliminate interfaces (`ElectricComponent` and `ElectricBreaker`) since interfaces cannot be declared as `final`. Next, we can look at the `ElectricCircuit` class and its subclasses (`ParallelCircuit`, `SeriesCircuit`, and `ShortCircuit`). Obviously, since `ElectricCircuit` has subclasses, it cannot be `final`.

However, its subclasses are modeling notions that shouldn't be extended, so they can be `final`. This is our first step in obtaining a closed hierarchical model:

```
public final class ParallelCircuit extends ElectricCircuit {}  
public final class SeriesCircuit extends ElectricCircuit {}  
public final class ShortCircuit extends ElectricCircuit {}
```

Other classes that model well-defined notions that shouldn't be extended are the classes that model capacitors, transistors, and resistors. So, the following classes can be `final` as well:

```
public final class CeramicCapacitor extends Capacitor {}  
public final class ElectrolyticCapacitor extends Capacitor {}  
public final class FieldEffectTransistor extends Transistor {}  
public final class BipolarTransistor extends Transistor {}  
public final class CarbonResistor extends Resistor {}  
public final class MetalFilmResistor extends MetalResistor {}  
public final class MetalOxideResistor extends MetalResistor {}
```

Finally, we have the `ElectricPanel` class. It doesn't make sense to derive something from an electrical panel, so this class can be `final` as well:

```
public final class ElectricPanel implements ElectricBreaker {}
```

So far, we managed to close some parts of the hierarchy. There are no other places where the `final` modifier can help us, so we can go further and try another technique.

Defining package-private constructors

Next, we can use the hack of defining *package-private* constructors (a constructor with no visible modifier). The classes having *package-private* constructors can be instantiated and extended only inside that package – from a readability point of view, this technique is far away from expressing its intentions. However, in complex designs, we can apply this technique spo-

radically since we cannot simply put everything in a single package. Nevertheless, it can be considered as a solution for increasing the hierarchical model closing level.

For instance, we can focus on our `abstract` classes. They cannot be instantiated (being `abstract`) but they can be extended from anywhere. However, some of them should be extended only in the package where they are defined. The `ElectricCircuit` class is `abstract` and it should be extended only by `ParallelCircuit`, `SeriesCircuit`, and `ShortCircuit`. These subclasses live in the same package as `ElectricCircuit`, so it makes sense to use this hack of declaring a *package-private* constructor:

```
public abstract class ElectricCircuit
    implements ElectricComponent {
    ElectricCircuit() {}
    ...
}
```

Now, the `ElectricCircuit` class is closed to any extension attempt coming from outside of its package. Of course, it is still open to extension attempts from inside of its package.

Declaring classes/interfaces as non-public

Going further, we can declare interfaces/classes as non-public (by skipping the `public` keyword from the class/interface definition, it becomes non-public and is set by default in the so-called *package-private* access mode). This way, those classes and interfaces are visible (can be used/extended) only inside their packages. We cannot apply this technique to the `ElectricComponent` interface. This interface has to be `public` because it is implemented by most of our classes. However, we can apply this technique to the `ElectricBreaker` interface, since this interface should be implemented only by the `ElectricPanel` class, which is in the same package as it:

```
interface ElectricBreaker extends ElectricComponent {}
```

Now, `ElectricBreaker` cannot be extended/implemented outside its package. Moreover, we can apply this technique to the `abstract` classes `Transistor`, `Resistor`, and `Capacitor`:

```
abstract class Capacitor implements ElectricComponent {}  
abstract class Resistor implements ElectricComponent {}  
abstract class Transistor implements ElectricComponent {}
```

Notice that we cannot apply this technique to the `ElectricCircuit` class. This class is `abstract` but it is used in the `ElectricPanel` class, so it cannot be non-public. However,

it cannot be extended thanks to the *package-private* constructor added previously.

Throwing everything in a module

In addition, we can place the whole hierarchy inside a Java module and export/expose to our client only a small part of it. However, this practice will not affect the closing level from inside the module, so we will skip it (that is, we will not exemplify it).

At this moment almost the entire hierarchy is closed to extension/implementation. The exceptions are the `MetalResistor` class and the `ElectricComponent` interface, which can be extended/implemented from anywhere inside/outside the model, and the `ElectricCircuit`, `Capacitor`, `Transistor`, and `Resistor` classes, which can be extended from inside of their packages. By placing the model in a Java module, we can block these actions from outside the module, but they are still possible from inside the module.

Conclusion

From this point forward (prior to JDK 17), there are no more techniques, tricks, or hacks that we can apply. We can reconsider the model design, but this will be too costly and will basi-

cally mean redesigning the model entirely, which may affect the model structure and logic.

For the sake of discussion and in the context of redesigning, we may consider Java enums. Java enums give us a nice closed hierarchy and are transformed internally in regular Java classes. Nevertheless, using enums to design a closed model and shape arbitrary classes can be really weird, unwieldy, and inconvenient.

In conclusion, before JDK 17, we had the radical `final` modifier and some control at the package level via *package-private* access.

It is obvious that what's missing here is something in between, something to give us more granularity and control. Fortunately, JDK 17 can help us to achieve a 100% closed hierarchy via sealed classes. This is the topic of some of the next problems.

174. Introducing JDK 17 sealed classes

Among the cool features of JDK 17, we have JEP 409 (sealed classes). This JEP provides an explicit, intuitive, crystal-clear solution for nominating who will extend a class/interface or

will implement an interface. In other words, sealed classes can control inheritance at a finer level. Sealed classes can affect classes, `abstract` classes, and interfaces and sustain the readability of the code – you have an easy and expressive solution to tell your colleagues who can extend/implement your code.



Figure 8.3: JDK 17, JEP 409

Via sealed classes, we have finer control over a hierarchy of classes. As you can see from the previous figure, sealed classes are the missing piece of the puzzle sitting between `final` and `package-private`. In other words, sealed classes provide a granularity that we cannot obtain via the `final` modifier and `package-private` access.

Important note

Sealed classes don't affect the semantics of the `final` and `abstract` keywords. They still act ex-

actly as they have for years. A sealed class cannot be `final` and vice versa.

Let's consider the following class (`Truck.java`):

```
public class Truck {}
```

We know that, in principle, this class can be extended by any other class. But we have only three types of trucks: semi-trailer, tautliner, and refrigerated. So, only three classes should extend the `Truck` class. Any other extension should not be allowed. In order to achieve this goal, we seal the class `Truck` by adding in its declaration the `sealed` keyword, as follows:

```
public sealed class Truck {}
```

By adding the `sealed` keyword, the compiler will automatically scan for all the extensions of `Truck` predefined in `Truck.java`.

Next, we have to specify the subclasses of `Truck` (`SemiTrailer`, `Tautliner`, and `Refrigerated`).

Important note



A `sealed` class (`abstract` or not) must have at least a subclass (otherwise there is no point in declaring it `sealed`). A `sealed` interface must have at least a subinterface or an implementation (again, otherwise there is no point declaring it `sealed`). If we don't follow these rules, then the code will not compile.

If we declare the subclasses of `Truck` in the same source file (`Truck.java`), then we can do so as follows:

```
final class SemiTrailer extends Truck {}  
final class Tautliner extends Truck {}  
final class Refrigerated extends Truck {}
```

After checking this code, we have to push another important note.

Important note

A subclass of a `sealed` class must be declared `final`, `sealed`, or `non-sealed`. A subinterface of a `sealed` interface must be declared `sealed` or `non-sealed`. If the subclass (subinterface) of a `sealed` class (interface) is declared as `sealed`, then it must have its own subclasses (subinter-





faces). The `non-sealed` keyword indicates that the subclass (subinterface) can be freely extended further with no restrictions (the hierarchy containing a `non-sealed` class/interface is not closed). And, a `final` subclass cannot be extended.

Since our subclasses (`SemiTrailer`, `Tautliner`, and `Refrigerated`) are declared `final`, they cannot be extended further. So, the `Truck` class can be extended only by `SemiTrailer`, `Tautliner`, and `Refrigerated`, and these classes are non-extendable.

In the case of interfaces, we do the same. For instance, a `sealed` interface looks like this:

```
public sealed interface Melon {}
```

By adding the `sealed` keyword, the compiler will automatically scan for all the implementations/extensions of `Melon` predefined in `Melon.java`. So, in the same source file (`Melon.java`), we declare the extensions and implementations of this interface:

```

non-sealed interface Pumpkin extends Melon {}
final class Gac implements Melon {}
final class Cantaloupe implements Melon {}
final class Hami implements Melon {}

```

The `Pumpkin` interface can be further freely implemented/extended since it is declared as `non-sealed`. The implementations/extensions of `Pumpkin` don't need to be declared `sealed`, `non-sealed`, or `final` (but we can still make this declaration).

Next, let's look at a more complex example. Let's name this model the *Fuel* model. Here, all classes and interfaces are placed in the same source file, `Fuel.java` (the `com.refinery.fuel` package). Take your time and analyze each class/interface to understand how `sealed`, `non-sealed`, and `final` work together in this hierarchal model:

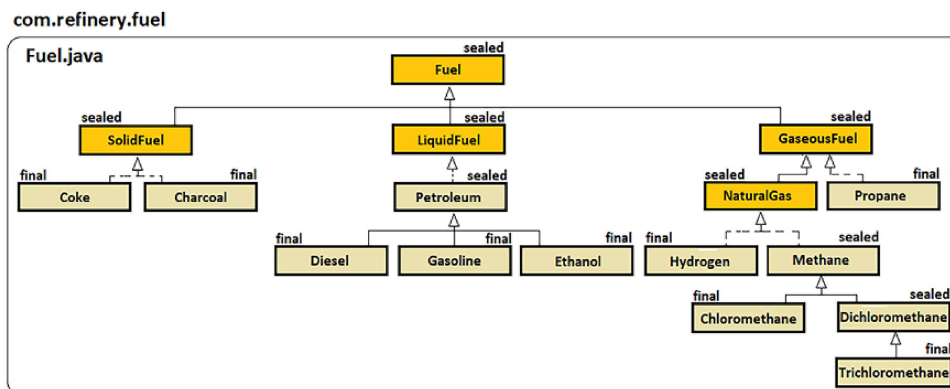


Figure 8.4: A hierarchical model using sealed, non-sealed, and final

In code lines, this model can be expressed as follows:

```
public sealed interface Fuel {}
sealed interface SolidFuel extends Fuel {}
sealed interface LiquidFuel extends Fuel {}
sealed interface GaseousFuel extends Fuel {}
final class Coke implements SolidFuel {}
final class Charcoal implements SolidFuel {}
sealed class Petroleum implements LiquidFuel {}
final class Diesel extends Petroleum {}
final class Gasoline extends Petroleum {}
final class Ethanol extends Petroleum {}
final class Propane implements GaseousFuel {}
sealed interface NaturalGas extends GaseousFuel {}
final class Hydrogen implements NaturalGas {}
sealed class Methane implements NaturalGas {}
final class Chloromethane extends Methane {}
sealed class Dichloromethane extends Methane {}
final class Trichloromethane extends Dichloromethane {}
```

Placing all the classes/interfaces in the same source file allows us to express closed hierarchical models like the previous one. However, placing all classes and interfaces in the same file is rarely a useful approach – maybe when the model contains a few small classes/interfaces.

In reality, we like to separate classes and interfaces into their own source files. It is more natural and intuitive to have each class/interface in its own source file. This way, we avoid large sources and it is much easier to follow the best practices of OOP. So, the goal of our next problem is to rewrite the *Fuel* hierarchical model by using a source file per class/interface.

175. Introducing the permits clause

In the previous problem, you saw how to write a closed hierarchical model in a single source file. Next, let's use the `Fuel.java` source file to rewrite this model by using separate sources and separate packages.

Working with sealed classes in separate sources (same package)

Let's consider the sealed `Fuel` interface from `Fuel.java` in package `com.refinery.fuel`:

```
public sealed interface Fuel {} // Fuel.java
```

We know that this interface is extended by three other interfaces: `SolidFuel`, `LiquidFuel`, and `SolidFuel`. Let's define

`SolidFuel` in the `SolidFuel.java` source (same package), as follows:

```
public sealed interface SolidFuel {} // SolidFuel.java
```

As you'll see, this code will not compile (it is like the compiler is asking: *hey, what's the point of a sealed interface without any implementation/extension?*). This time, we have to explicitly nominate the interfaces that can extend/implement the `Fuel` interface. For this, we use the `permits` keyword. Since `Fuel` is implemented by three interfaces, we just list their names via `permits` as follows:

```
public sealed interface Fuel
    permits SolidFuel, LiquidFuel, GaseousFuel {}
```

The list provided via `permits` is exhaustive. The `SolidFuel` is also a `sealed` interface, so it has to define its `permits` as well:

```
public sealed interface SolidFuel extends Fuel
    permits Coke, Charcoal {}
```

`LiquidFuel` and `GaseousFuel` work the same way as `SolidFuel`:

```
// LiquidFuel.java
public sealed interface LiquidFuel extends Fuel
    permits Petroleum {}
// GaseousFuel.java
public sealed interface GaseousFuel extends Fuel
    permits NaturalGas, Propane {}
```

The `Coke` (`Coke.java`) and `Charcoal` (`Charcoal.java`) are final implementations of `SolidFuel`, so they don't use the `permits` keyword:

```
public final class Coke implements SolidFuel {}
public final class Charcoal implements SolidFuel {}
```

The `Petroleum` class (`Petroleum.java`) is `sealed` and allows three extensions:

```
public sealed class Petroleum implements LiquidFuel
    permits Diesel, Gasoline, Ethanol {}
```

The `Diesel` (`Diesel.java`), `Gasoline` (`Gasoline.java`), and `Ethanol` (`Ethanol.java`) classes are `final`:

```
public final class Diesel extends Petroleum {}
public final class Gasoline extends Petroleum {}
```

```
public final class Ethanol extends Petroleum {}
```

The `NaturalGas` interface (`NaturalGas.java`) is a sealed extension of `GaseousFuel`, while `Propane` (`Propane.java`) is a final implementation of `GaseousFuel`:

```
public sealed interface NaturalGas extends GaseousFuel
    permits Hydrogen, Methane {}
public final class Propane implements GaseousFuel {}
```

As you can see, the `NaturalGas` interface permits two extensions. The `Hydrogen` class is a final extension, while `Methane` is a sealed class:

```
public final class Hydrogen implements NaturalGas {}
public sealed class Methane implements NaturalGas
    permits Chloromethane, Dichloromethane {}
```

The `Chloromethane` class is final, and `Dichloromethane` is sealed:

```
public final class Chloromethane extends Methane {}
public sealed class Dichloromethane extends Methane
    permits Trichloromethane {}
```

Finally, we have the `Trichloromethane` class. This is a `final` class:

```
public final class Trichloromethane extends Dichloromethane {}
```

Done! The hierarchical model is closed and complete. Any attempt to extend/implement any member of this hierarchy will lead to an exception. If we want to add a new extension/implementation to a `sealed` class/interface, then we have to add it to the `permits` list as well.

Working with sealed classes in separate packages

In the previous example, we expressed the classes/interfaces in separate sources but in the same package,

`com.refinery.fuel`. Next, let's consider that we spread these classes and interfaces across different packages as in the following figure:

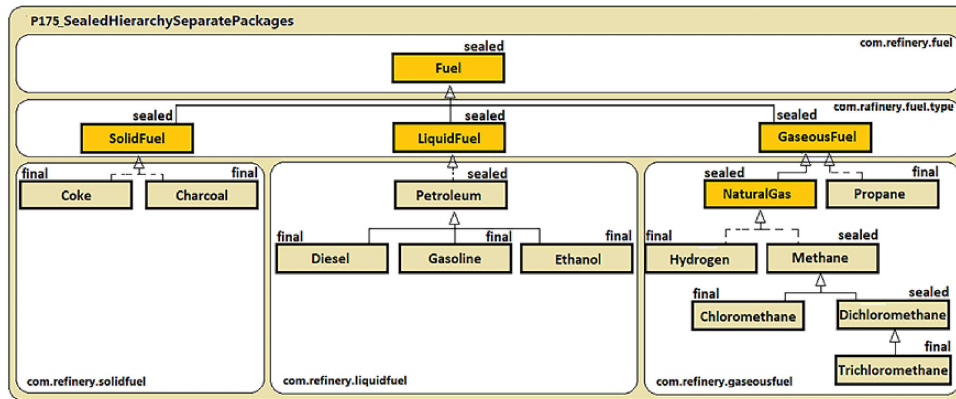


Figure 8.5: Sealed hierarchy in different packages

As long as the related `sealed` classes/interfaces live in the same package, we can use the JDK 9 *unnamed* special module (no explicit module). Otherwise, we have to use a *named* module. For instance, if we express our model as in Figure 8.5, then we have to add everything in a module via `module-info.java`:

```
module P175_SealedHierarchySeparatePackages {}
```

Without a *named* module, the code will not compile. In the bundled code, you can find both examples from this problem.

176. Closing the electrical panel after JDK 17

Do you remember our electrical panel model introduced earlier in *Problems 172* and *173*? In *Problem 173*, we closed this model as much as possible by using the Java capabilities available before JDK 17. Now, we can revisit that model (*Problem 173*) and close it completely via JDK 17 sealed classes.

We start with the `ElectricComponent` interface, which is declared as follows:

```
public interface ElectricComponent {}
```

At this moment, this interface is not closed. It can be extended/implemented from any other point of the application. But we can close it by transforming it into a `sealed` interface with the proper `permits` clause, as follows:

```
public sealed interface ElectricComponent
    permits ElectricCircuit, ElectricBreaker,
           Capacitor, Resistor, Transistor {}
```

Next, let's focus on the semi-closed `ElectricCircuit` class. This is an `abstract` class that uses a *package-private* constructor to block any extension from outside its package. However, it can still be extended from inside the package. We can close it completely by transforming it into a `sealed` class with the

proper `permits` clause (the *package-private* constructor can be safely removed):

```
public sealed abstract class ElectricCircuit
    implements ElectricComponent
    permits ParallelCircuit, SeriesCircuit, ShortCircuit {}
```

The `ParallelCircuit`, `SeriesCircuit`, and `ShortCircuit` were declared as `final`, so they remain unchanged. We don't want to permit any extension of these classes.

Next, let's focus on the `Capacitor`, `Transistor`, and `Resistor` classes. These classes are also `abstract` and use *package-private* constructors to avoid any extension attempts coming from outside of their packages. So, we can remove these constructors and transform them into `sealed` classes exactly as we did with `ElectricCircuit`:

```
public sealed abstract class Capacitor
    implements ElectricComponent
    permits CeramicCapacitor, ElectrolyticCapacitor {}
public sealed abstract class Transistor
    implements ElectricComponent
    permits FieldEffectTransistor, BipolarTransistor {}
public sealed abstract class Resistor
    implements ElectricComponent
    permits MetalResistor, CarbonResistor {}
```

Check out the `Resistor` class. It permits only `MetalResistor` and `CarbonResistor` classes. Next, the `MetalResistor` class needs special attention. So far, this class is `public` and can be extended from any other point of the application:

```
public class MetalResistor extends Resistor {}
```

Closing this class can be done by sealing it as follows:

```
public sealed class MetalResistor extends Resistor  
    permits MetalFilmResistor, MetalOxideResistor {}
```

The `MetalFilmResistor` and `MetalOxideResistor` classes are `final` and remain unchanged:

```
public final class MetalFilmResistor extends MetalResistor {}  
public final class MetalOxideResistor extends MetalResistor {}
```

The same statement applies to the `CeramicCapacitor`, `ElectrolyticCapacitor`, `BipolarTransistor`, and `FieldEffectTransistor` classes.

Next, let's focus on the `ElectricBreaker` interface. This interface lives in the `modern.circuit.panel` package and was implemented only by `ElectricPanel`, so it was declared *pack-*

age-private (it cannot be extended/implemented from outside the package):

```
interface ElectricBreaker extends ElectricComponent {}
```

In order to completely close this interface, we transform it into a `sealed` interface, as follows:

```
public sealed interface ElectricBreaker  
    extends ElectricComponent permits ElectricPanel {}
```

Notice that we added the `public` modifier as well. This is needed because `ElectricBreaker` must occur in the `permits` list of the `ElectricComponent` interface, so it has to be available outside its package.

Finally, the `ElectricPanel` remains unchanged (a `final` class implementing `ElectricBreaker`):

```
public final class ElectricPanel implements ElectricBreaker {}
```

Mission accomplished! The electric panel hierarchical model is completely closed to extension. We put everything in a *named* module (since we have `sealed` artifacts that interact across different packages) and we are done.

177. Combining sealed classes and records

As you know from *Chapter 4*, Java records are `final` classes that cannot be extended and cannot extend other classes. This means that records and `sealed` classes/interfaces can team up to obtain a closed hierarchy.

For instance, in the following figure, we can identify the classes that can be good candidates to become Java records in the *Fuel* model:

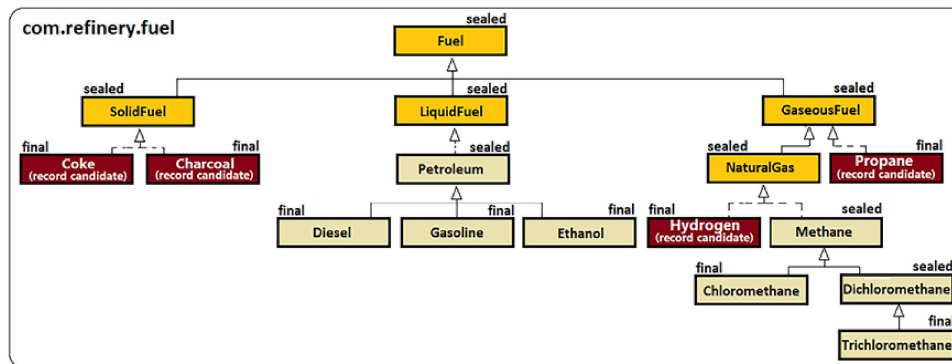


Figure 8.6: Identify classes that can become Java records

As you can see, we have four classes that can become Java records: `Coke`, `Charcoal`, `Hydrogen`, and `Propane`.

Technically speaking, these classes can be Java records since they are `final` classes and don't extend other classes:

```
public record Coke() implements SolidFuel {}  
public record Charcoal() implements SolidFuel {}  
public record Hydrogen() implements NaturalGas {}  
public record Propane() implements GaseousFuel {}
```

Of course, the technical aspect is important but it is not enough. In other words, you don't have to transform all classes into Java records just because it works and the code compiles. You also have to take into account the logic and the context of the application. Sometimes, a `final` class is all you need; otherwise, you may need an esoteric model made of a `sealed` interface with a few records and classes in the same source file (`A.java`):

```
public sealed interface A {  
    record A1() implements A {}  
    record A2() implements A {}  
    final class B1 implements A {}  
    non-sealed class B2 implements A {}  
}  
record A3() implements A {}  
record A4() implements A {}
```

If you want to add the `permits` clause to `A`, then you can do it as follows:

```
public sealed interface A
    permits A.A1, A.A2, A.B1, A.B2, A3, A4 {...}
```

Done! Next, let's see how sealed classes can help the compiler to better handle `instanceof` checks.

178. Hooking sealed classes and `instanceof`

Sealed classes influence how the compiler understands the `instanceof` operator and, implicitly, how it performs internal cast and conversion operations.

Let's consider the following snippet of code:

```
public interface Quadrilateral {}
public class Triangle {}
```

So, we have here an interface (`Quadrilateral`) and a class that doesn't implement this interface. In this context, does the following code compile?

```
public static void drawTriangle(Triangle t) {
    if (t instanceof Quadrilateral) {
        System.out.println("This is not a triangle");
    } else {
```

```
        System.out.println("Drawing a triangle");  
    }  
}
```

We wrote `if (t instanceof Quadrilateral) {...}` but we know that `Triangle` doesn't implement `Quadrilateral`, so at first sight, we may think that the compiler will complain about this. But, actually, the code compiles because, at runtime, we may have a class `Rectangle` that extends `Triangle` and implements `Quadrilateral`:

```
public class Rectangle extends Triangle  
    implements Quadrilateral {}
```

So, our `instanceof` makes sense and is perfectly legal. Next, let's close the `Triangle` class via the `final` keyword:

```
public final class Triangle {}
```

Since `Triangle` is `final`, `Rectangle` cannot extend it, but it can still implement `Quadrilateral`:

```
public class Rectangle implements Quadrilateral {}
```

This time, the `if (t instanceof Quadrilateral) {...}` code will not compile. The compiler knows that a `final` class cannot be extended, so a `Triangle` will never be a `Quadrilateral`.

So far, so good! Now, let's restore the `Triangle` class as a non-`final` class:

```
public class Triangle {}
```

And let's seal the `Quadrilateral` interface to permit only `Rectangle`:

```
public sealed interface Quadrilateral permits Rectangle {}
```

And, the `Rectangle` class is `final`, as follows (this time, it doesn't extend `Triangle`):

```
public final class Rectangle implements Quadrilateral {}
```

Again, the compiler will complain about this check, `if (t instanceof Quadrilateral) {...}`. It is obvious that `Triangle` cannot be an instance of `Quadrilateral` since `Quadrilateral` is sealed and permits only `Rectangle`, not

`Triangle`. However, if we modify the `Rectangle` to extend `Triangle`, then the code compiles:

```
public final class Rectangle extends Triangle
    implements Quadrilateral {}
```

So, in conclusion, sealed classes can help the compiler to better understand `instanceof` checks and to signal us when it doesn't make sense.

179. Hooking sealed classes in `switch`

This is not the first time in this book that we've presented an example of sealed classes and `switch` expressions. In *Chapter 2, Problem 66*, we briefly introduced such an example via the sealed `Player` interface with the goal of covering completeness (type coverage) in pattern labels for `switch`.

If, at that time, you found this example confusing, I'm pretty sure that now it is clear. However, let's keep things fresh and see another example starting from this `abstract` base class:

```
public abstract class TextConverter {}
```

And, we have three converters available, as follows:

```
final public class Utf8 extends TextConverter {}  
final public class Utf16 extends TextConverter {}  
final public class Utf32 extends TextConverter {}
```

Now, we can write a `switch` expression to match these `TextConverter` instances, as follows:

```
public static String convert(  
    TextConverter converter, String text) {  
    return switch (converter) {  
        case Utf8 c8 -> "Converting text to UTF-8: " + c8;  
        case Utf16 c16 -> "Converting text to UTF-16: " + c16;  
        case Utf32 c32 -> "Converting text to UTF-32: " + c32;  
        case TextConverter tc -> "Converting text: " + tc;  
        default -> "Unrecognized converter type";  
    };  
}
```

Check out the highlighted lines of code. After the three cases (`case Utf8`, `case Utf16`, and `case Utf32`), we must have one of the `case TextConverter` or the `default` case. In other words, after matching `Utf8`, `Utf16`, and `Utf32`, we must have a total type pattern (unconditional pattern) to match any other `TextConverter` or a `default` case, which typically means that we are facing an unknown converter.

If both the total type pattern and the `default` label are missing, then the code doesn't compile. The `switch` expression doesn't cover all the possible cases (input values), therefore it is not exhaustive. This is not allowed, since `switch` expressions and `switch` statements that use `null` and/or pattern labels should be exhaustive.

The compiler will consider our `switch` as non-exhaustive because we can freely extend the base class (`TextConverter`) with uncovered cases. An elegant solution is to seal the base class (`TextConverter`) as follows:

```
public sealed abstract class TextConverter
    permits Utf8, Utf16, Utf32 {}
```

And now the `switch` can be expressed as follows:

```
return switch (converter) {
    case Utf8 c8 -> "Converting text to UTF-8: " + c8;
    case Utf16 c16 -> "Converting text to UTF-16: " + c16;
    case Utf32 c32 -> "Converting text to UTF-32: " + c32;
};
```

This time, the compiler knows all the possible `TextConverter` types and sees that they are all covered in the `switch`. Since `TextConverter` is `sealed`, there are no surprises; no uncov-

ered cases can occur. Nevertheless, if we later decide to add a new `TextConverter` (for instance, we add `Utf7` by extending `TextConverter` and adding this extension in the `permits` clause), then the compiler will immediately complain that the `switch` is non-exhaustive, so we must take action and add the proper `case` for it.

At this moment, `Utf8`, `Utf16`, and `Utf32` are declared as `final`, so they cannot be extended. Let's assume that `Utf16` is modified to become `non-sealed`:

```
non-sealed public class Utf16 extends TextConverter {}
```

Now, we can extend `Utf16` as follows:

```
public final class Utf16be extends Utf16 {}  
public final class Utf16le extends Utf16 {}
```

Even if we added two subclasses to the `Utf16` class, our `switch` is still exhaustive because the case `Utf16` will cover `Utf16be` and `Utf16le` as well. Nevertheless, we can explicitly add cases for them, as long as we add these cases before `case` `Utf16`, as follows:

```
return switch (converter) {  
    case Utf8 c8 -> "Converting text to UTF-8: " + c8;  
    case Utf16be c16 -> "Converting text to UTF-16BE: " + c16;  
    case Utf16le c16 -> "Converting text to UTF-16LE: " + c16;  
    case Utf16 c16 -> "Converting text to UTF-16: " + c16;  
    case Utf32 c32 -> "Converting text to UTF-32: " + c32;  
};
```

We have to add `case Utf16be` and `case Utf16le` before `case Utf16` to avoid dominance errors (see *Chapter 2, Problem 65*).

Here is another example of combining sealed classes, pattern matching for switch, and Java records to compute the sum of nodes in a binary tree of integers:

```
sealed interface BinaryTree {  
    record Leaf() implements BinaryTree {}  
    record Node(int value, BinaryTree left, BinaryTree right)  
        implements BinaryTree {}  
}  
static int sumNode(BinaryTree t) {  
    return switch (t) {  
        case Leaf nl -> 0;  
        case Node nv -> nv.value() + sumNode(nv.left())  
            + sumNode(nv.right());  
    }  
}
```

```
};  
}
```

And here is an example of calling `sumNode()` :

```
BinaryTree leaf = new Leaf();  
BinaryTree s1 = new Node(5, leaf, leaf);  
BinaryTree s2 = new Node(10, leaf, leaf);  
BinaryTree s = new Node(4, s1, s2);  
int sum = sumNode(s);
```

In this example, the result is `19` .

180. Reinterpreting the Visitor pattern via sealed classes and type pattern matching for switch

The Visitor pattern is part of the **Gang of Four (GoF)** design patterns and its goal is to define a new operation on certain classes without the need to modify those classes. You can find many excellent resources on this topic on the Internet, so for the classical implementation, we will provide here only the class diagram of our example, while the code is available on GitHub:

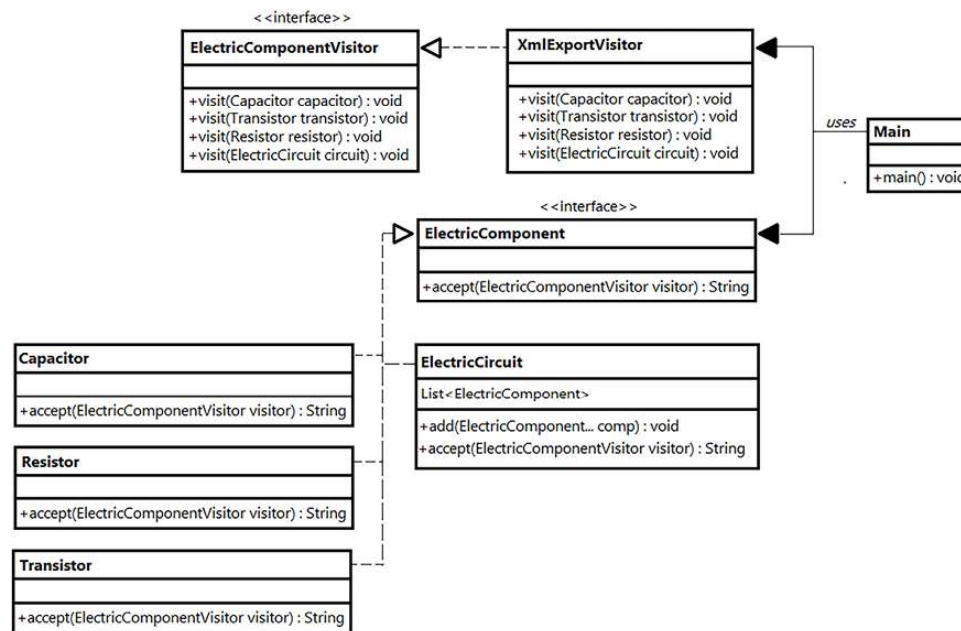


Figure 8.7: Visitor pattern class diagram (use case)

In a nutshell, we have a bunch of classes (`Capacitor` , `Transistor` , `Resistor` , and `ElectricCircuit`) that are used to create electrical circuits. Our operation is shaped in `XmlExportVisitor` (an implementation of `ElectricComponentVisitor`) and consists of printing an XML document containing the electrical circuit specifications and parameters.

Before continuing, consider getting familiar with the traditional implementation and output of this example available in the bundled code.

Next, let's assume that we want to transform this traditional implementation via sealed classes and type pattern matching for `switch`. The expected class diagram is simpler (has fewer classes) and it looks as follows:

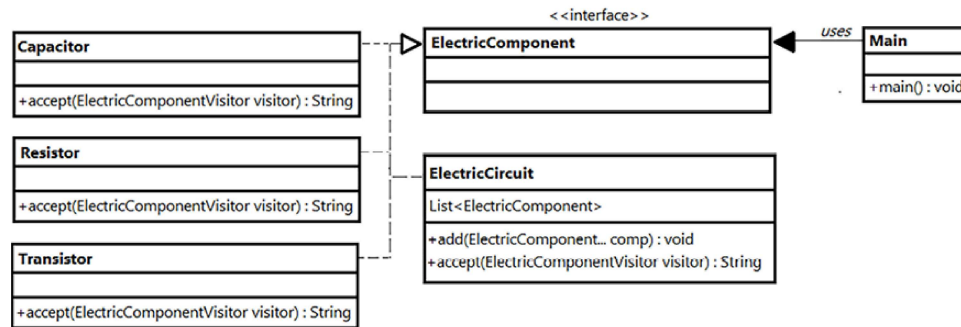


Figure 8.8: Visitor pattern reinterpreted via sealed classes and switch patterns

Let's start the transformation with the `ElectricComponent` interface. We know that this interface is implemented only by `Capacitor`, `Resistor`, `Transistor`, and `ElectricCircuit`. So, this interface is a good candidate to become `sealed`, as follows:

```
public sealed interface ElectricComponent
    permits Capacitor, Transistor, Resistor, ElectricCircuit {}
```

Notice that we deleted the `accept()` method from this interface. We no longer need this method. Next, the `Capacitor`, `Resistor`, `Transistor`, and `ElectricCircuit` become `final` classes and the `accept()` implementation is deleted as well.

Since we don't rely on the traditional Visitor pattern, we can safely remove its specific artifacts such as `ElectricComponentVisitor` and `XmlComponentVisitor`.

Pretty clean, right? We remained with a `sealed` interface and four `final` classes. Next, we can write a `switch` that visits each component of a circuit, as follows:

```
private static void export(ElectricComponent circuit) {  
    StringBuilder sb = new StringBuilder();  
    sb.append("<?xml version=\"1.0\" encoding=\"utf-8\"?>\n");  
    export(sb, circuit);  
    System.out.println(sb);  
}
```

The `export(StringBuilder sb, ElectricComponent... comps)` is the effective visitor:

```
private static String export(StringBuilder sb,  
    ElectricComponent... comps) {  
    for (ElectricComponent comp : comps) {
```

```
switch (comp) {
case Capacitor c ->
    sb.append("""
        <capacitor>
            <maxImpedance>%s</maxImpedance>
            <dielectricResistance>%s</dielectricResistance>
            <coreTemperature>%s</coreTemperature>
        </capacitor>
        """).formatted(c.getMaxImpedance(),
            c.getDielectricResistance(),
            c.getCoreTemperature()));
case Transistor t ->
    sb.append("""
        <transistor>
            <length>%s</length>
            <width>%s</width>
            <threshholdVoltage>%s</threshholdVoltage>
        </transistor>
        """).formatted(t.getLength(), t.getWidth(),
            t.getThreshholdVoltage()));
case Resistor r ->
    sb.append("""
        <resistor>
            <resistance>%s</resistance>
            <clazz>%s</clazz>
            <voltage>%s</voltage>
            <current>%s</current>
            <power>%s</power>
        </resistor>
        """).formatted(r.getResistance(), r.getClazz(),
```

```
                r.getVoltage(), r.getCurrent(),
                r.getPower()).toString();
    case ElectricCircuit ec ->
        sb.append(""  
            <electric_circuit_%s>  
            %s\  
            </electric_circuit_%s>  
            ""formatted(ec.getId(),  
                export(new StringBuilder(),  
                    ec.getComps().toArray(ElectricComponent[]::new)),  
                    ec.getId()).indent(3)).toString();  
        }  
    }  
    return sb.toString();  
}
```

Mission accomplished! You can find the complete example in the bundled code.

181. Getting info about sealed classes (using reflection)

We can inspect sealed classes via two methods added as part of the Java Reflection API. First, we have `isSealed()`, which is a flag method useful to check if a class is or isn't sealed.

Second, we have `getPermittedSubclasses()`, which returns an array containing the permitted classes. Based on these two

methods, we can write the following helper to return the permitted classes of a `sealed` class:

```
public static List<Class> permittedClasses(Class clazz) {  
    if (clazz != null && clazz.isSealed()) {  
        return Arrays.asList(clazz.getPermittedSubclasses());  
    }  
    return Collections.emptyList();  
}
```

We can easily test our helper via the `Fuel` model as follows:

```
Coke coke = new Coke();  
Methane methane = new Methane();  
// [interface com.refinery.fuel.SolidFuel,  
// interface com.refinery.fuel.LiquidFuel,  
// interface com.refinery.fuel.GaseousFuel]  
System.out.println("Fuel subclasses: "  
    + Inspector.permittedClasses(Fuel.class));  
// [class com.refinery.fuel.Coke,  
// class com.refinery.fuel.Charcoal]  
System.out.println("SolidFuel subclasses: "  
    + Inspector.permittedClasses(SolidFuel.class));  
// []  
System.out.println("Coke subclasses: "  
    + Inspector.permittedClasses(coke.getClass()));  
// [class com.refinery.fuel.Chloromethane,  
// class com.refinery.fuel.Dichloromethane]
```

```
System.out.println("Methane subclasses: "  
    + Inspector.permittedClasses(methane.getClass()));
```

I think you got the idea!

182. Listing the top three benefits of sealed classes

Maybe you have your own top three sealed class benefits that don't match the following list. That's OK, they are still benefits after all!

- **Sealed classes sustain better design and clearly expose their intentions:** Before using sealed classes, we have to rely only on the `final` keyword (which is expressive enough), and *package-private* classes/constructors. Obviously, *package-private* code requires some reading between the lines to understand its intention since it is not easy to spot a closed hierarchy modeled via this hack. On the other hand, sealed classes expose their intentions very clearly and expressively.
- **The compiler can rely on sealed classes to perform finer checks on our behalf:** Nobody can sneak a class into a hierarchy closed via sealed classes. Any such attempt is rejected via a clear and meaningful message. The compiler is guarding for us and acts as the first line of defense against

any accidental/non-accidental attempt to use our closed hierarchies in an improper way.

- **Sealed classes help the compiler to provide better pattern matching:** You experimented with this benefit in *Problem 179*. The compiler can rely on sealed classes to determine if a `switch` is covering all the possible input values and therefore is exhaustive. And this is just the beginning of what sealed classes can do for pattern matching.

183. Briefly introducing hidden classes

Hidden classes were introduced in JDK 15 under JEP 371. Their main goal is to be used by frameworks as dynamically generated classes. They are runtime-generated classes with a short lifespan that are used by frameworks via reflection.

Important note



Hidden classes cannot be used directly by bytecode or other classes. They are not created via a class loader. Basically, a hidden class has the class loader of the lookup class.

Among other characteristics of hidden classes, we should consider that:

- They are not discoverable by the JVM internal linkage of bytecode or by the explicit usage of class loaders (they are invisible to methods such as `Class.forName()`, `Lookup.findClass()`, or `ClassLoader.findLoadedClass()`). They don't appear in stack traces.
- They extend **Access Control Nest (ACN)** with classes that cannot be discovered.
- Frameworks can define hidden classes, as many as needed, since they benefit from aggressive unloading. This way, a large number of hidden classes shouldn't have a negative impact on performance. They sustain efficiency and flexibility.
- They cannot be used as a field/return/parameter type. They cannot be superclasses.
- They can access their code directly without the presence of a class object.
- They can have `final` fields, and those fields cannot be modified regardless of their accessible flags.
- They deprecated the `misc.Unsafe::defineAnonymousClass`, which is a non-standard API. Starting with JDK 15, lambda expressions use hidden classes instead of anonymous classes.

Next, let's see how we can create and use a hidden class.

184. Creating a hidden class

Let's assume that our hidden class is named `InternalMath` and is as simple, as follows:

```
public class InternalMath {  
    public long sum(int[] nr) {  
        return IntStream.of(nr).sum();  
    }  
}
```

As we mentioned in the previous problem, hidden classes have the same class loader as the lookup class, which can be obtained via `MethodHandles.lookup()`, as follows:

```
MethodHandles.Lookup lookup = MethodHandles.lookup();
```

Next, we must know that `Lookup` contains a method named `defineHiddenClass(byte[] bytes, boolean initialize, ClassOption... options)`. The most important argument is represented by the array of bytes that contain the class data. The `initialize` argument is a flag specifying if the hidden class should be initialized or not, while the `options` argument can be `NESTMATE` (the created hidden class becomes a nest-mate of the lookup class and has access to all the private mem-

bers in the same nest) or `STRONG` (the created hidden class can be unloaded only if its defining loader is not reachable).

So, our goal is to obtain the array of bytes that contains the class data. For this, we rely on `getResourceAsStream()` and JDK 9's `readAllBytes()`, as follows:

```
Class<?> clazz = InternalMath.class;
String clazzPath = clazz.getName()
    .replace('.', '/') + ".class";
InputStream stream = clazz.getClassLoader()
    .getResourceAsStream(clazzPath);
byte[] clazzBytes = stream.readAllBytes();
```

Having `clazzBytes` in our hands, we can create the hidden class as follows:

```
Class<?> hiddenClass = lookup.defineHiddenClass(clazzBytes,
    true, ClassOption.NESTMATE).lookupClass();
```

Done! Next, we can use the hidden class from inside our framework, as follows:

```
Object obj = hiddenClass.getConstructor().newInstance();
Method method = obj.getClass()
    .getDeclaredMethod("sum", int[].class);
```

```
System.out.println(method.invoke(  
    obj, new int[] {4, 1, 6, 7})); // 18
```

As you can see, we use the hidden class via reflection. The interesting part here is represented by the fact that we cannot cast the hidden class to `InternalMath`, so we use `Object obj = ...`. So, this will not work:

```
InternalMath obj = (InternalMath) hiddenClass  
    .getConstructor().newInstance();
```

However, we can define an interface implemented by the hidden class:

```
public interface Math {}  
public class InternalMath implements Math {...}
```

And, now we can cast to `Math`:

```
Math obj = (Math) hiddenClass.getConstructor().newInstance();
```

Starting with JDK 16, the `Lookup` class was enriched with another method for defining a hidden class named

```
defineHiddenClassWithClassData(byte[] bytes, Object  
classData, boolean initialize, ClassOption... op-
```

tions). This method needs the class data obtained via `MethodHandles.classData(Lookup caller, String name, Class<T> type)` or `MethodHandles.classDataAt(Lookup caller, String name, Class<T> type, int index)`. Take your time to explore this further.

Summary

This chapter covered 13 problems. Most of them were focused on the sealed classes feature. The last two problems provided brief coverage of hidden classes.

Leave a review!

Enjoying this book? Help readers like you by leaving an Amazon review. Scan the QR code below for a 20% discount code.



**Limited Offer*

